

토큰화가 세계를 자르고 희소 벡터가 단서를 센다

문자열 → 토큰 → 빈도 → TF-IDF/BM25 → 혼합 검색

오늘의 중심 주장

핵심

토큰화는 전처리가 아니라 **모델이 볼 수 있는 최소 단위를 정하는 가설**이다.

- “목포대학교”, “목포”, “대학교” 중 무엇이 한 단위인가?
- “신청하지 않았다”에서 부정은 유지되는가?
- 희귀한 학번·법령명·제품 코드는 밀집 임베딩보다 희소 검색에 유리할 수 있다.

학습목표

1. token, vocabulary, OOV, subword, byte의 의미를 설명한다.
2. BPE·WordPiece·Unigram의 목표와 차이를 구분한다.
3. one-hot, BoW, TF-IDF, BM25를 직접 계산한다.
4. 희소·밀집 표현의 오류 양상을 비교하고 hybrid 검색을 설계한다.
5. Hands-On LLM 2장의 tokenizer 비교 코드를 한국어 예제로 재현한다.

용어 지도: 문자열에서 ID까지

문자열 (text) → 정규화 (normalize) → 분절 (pre-tokenize) → 서브워드 (model) → 정수 ID (encode)

토큰(token): 모델이 한 위치에서 처리하는 기호 단위.

어휘집(vocabulary): 토큰과 정수 ID의 유한한 대응표.

OOV(out-of-vocabulary): 어휘집에 없는 입력 단위.

토큰은 언어학적 “단어”와 같지 않으며, tokenizer 버전도 모델의 일부다.

토큰화의 선택지

단위	장점	약점
단어/형태소	해석이 쉬움	신조어·복합어·형태 변이, 분석기 의존
문자	OOV가 거의 없음	시퀀스가 길고 의미 단위가 잘게 쪼개짐
subword	길이·OOV의 절충	분절이 언어·데이터 빈도에 민감
byte	모든 문자열 표현 가능	토큰 수 증가, 사람이 읽기 어려움

주의

한국어: 조사·어미, 띄어쓰기 변이, 한글/영문/숫자 혼용 때문에 영어 tokenizer의 토큰 효율과 의미 보존이 그대로 유지되지 않는다.

BPE: 가장 빈번한 쌍을 합친다

간단한 학습 절차:

1. 단어를 문자(또는 byte) 단위로 시작한다.
2. 말뭉치에서 인접 기호 쌍의 빈도를 센다.
3. 가장 빈번한 쌍을 새 토큰으로 합친다.
4. 목표 어휘 크기까지 반복한다.

수식

학 생 들 → merge(학 , 생) → 학생 들 → merge(학생 , 들) → 학생들이

- **BPE**는 결정적인 merge 규칙 목록을 배운다.
- 빈도가 높은 문자열은 긴 토큰, 희귀 문자열은 작은 조각으로 표현된다.
- 실제 구현은 공백 표식, 정규화, byte fallback 등 세부이 다르다.

WordPiece와 Unigram

WordPiece

후보 결합이 언어모델 우도 개선에 얼마나 기여하는지 고려하는 계열. BERT tokenizer로 널리 알려짐.

예: `playing` → `play`, `##ing`

Unigram LM

큰 후보 어휘에서 시작해 우도를 덜 해치는 토큰을 제거. 한 문자열에 여러 분절 후보를 확률로 비교.

SentencePiece에서 사용 가능

참고: 이름만으로 구현을 단정하지 말고 tokenizer 설정 파일의 normalizer, pre-tokenizer, model, post-processor를 확인한다.

토큰화 품질을 어떻게 측정할까

fertility	단어/문장당 생성 토큰 수
coverage	문자·도메인·언어를 손실 없이 표현하는가
downstream	검색·분류 성능과 비용이 어떻게 변하는가

추가 진단:

- 평균만 보지 말고 언어·길이·고유명사별 분포를 본다.
- 같은 context window에서 토큰 수가 많으면 실제 담을 수 있는 내용이 줄어든다.
- **[UNK]** , 잘림(truncation), 정규화 손실을 별도로 집계한다.

one-hot과 Bag of Words

어휘 크기 $|V|$ 가 5라면 토큰 하나의 one-hot은

$$\text{학교} = [0, 1, 0, 0, 0]$$

문서 d 의 BoW 벡터는 각 토큰 빈도:

$$\mathbf{x}_d = [c(t_1, d), \dots, c(t_{|V|}, d)]$$

- 장점: 해석 가능, 정확한 키워드 보존, 빠른 inverted index
- 약점: 동의어 연결 부족, 어순·장거리 문맥 손실, 어휘가 클수록 고차원

정의

희소 벡터(sparse vector): 대부분 좌표가 0인 벡터. 실제 저장은 0이 아닌 인덱스와 값만 보관한다.

TF-IDF: 문서 안에서 중요하고, 말뭉치에서 드문 단어

한 가지 대표 정의:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N + 1}{\text{df}(t) + 1}$$

기호	뜻
N	전체 문서 수
$\text{tf}(t, d)$	문서 d 안의 토큰 t 빈도
$\text{df}(t)$	토큰 t 가 등장한 문서 수

주의

라이브러리마다 log, smoothing, sublinear TF, L2 normalization이 다르다. “TF-IDF”라는 이름만 같고 값은 다를 수 있다.

손계산: IDF가 하는 일

문서 4개 중

- **학교** 가 4개 문서에 등장: $\log((4 + 1)/(4 + 1)) = 0$
- **장학금** 이 1개 문서에 등장: $\log((4 + 1)/(1 + 1)) \approx 0.916$

공통어는 구별력이 낮고 희귀어는 구별력이 높다.

실습

토론: 희귀한 오타도 높은 IDF를 얻는다. 이것은 장점인가, 단점인가? 어떤 정규화/사전/최소 문서빈도로 제어할까?

BM25: 빈도 포화와 문서 길이 보정

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/\text{avgdl})}$$

- 같은 단어를 100번 쓴다고 점수가 100배 되지 않는 **TF saturation**
- 긴 문서는 단어가 우연히 많이 등장하므로 **length normalization**
- k_1 : 포화 속도, b : 길이 보정 강도

논문 읽기

BM25는 고전적이지만 여전히 강한 검색 기준선이다. 특히 ID·고유명사·희귀 키워드가 중요한 데이터에서 필요하다.

희소 vs 밀집: 서로 다른 실패를 한다

쿼리	희소 검색	밀집 검색
“근로장학 B-17 서식”	정확한 코드에 강함	코드가 희석될 수 있음
“등록금을 나눠 낼 수 있나요?”	“분할납부”와 lexical gap	의미 대응 가능
“신청하지 않은 학생”	부정 토큰에 민감	부정을 놓칠 수 있음
최신 법령명	즉시 인덱싱 가능	학습 데이터에 없을 수 있음

핵심

둘 중 하나가 항상 우월한 것이 아니라 **오류의 상관관계가 낮아 결합 가치가 크다.**

Hybrid retrieval

점수 정규화 후 가중합:

$$s_{hybrid}(q, d) = \alpha \tilde{s}_{dense}(q, d) + (1 - \alpha) \tilde{s}_{sparse}(q, d)$$

또는 순위만 결합하는 Reciprocal Rank Fusion:

$$\text{RRF}(d) = \sum_r \frac{1}{k + \text{rank}_r(d)}$$

- 서로 다른 점수 범위를 그대로 더하면 안 된다.
- α, k 는 validation query로 정한다.
- metadata filter는 결합 전/후 어느 단계에 적용하는지 기록한다.

Hands-On LLM 연결

tokenizer 비교에서 “토큰 수”와 “의미 손실”을 동시에 본다

원본 2장 실험을 한국어로 확장

```
texts = [  
    "국립목포대학교 장학금 신청 안내",  
    "AI·SW융합교육원 2026-2 프로그램",  
    "신청하지 않은 학생은 제외합니다",  
]  
  
for name, tokenizer in tokenizers.items():  
    ids = tokenizer(texts, add_special_tokens=False)["input_ids"]  
    print(name, [len(x) for x in ids])  
    print([tokenizer.convert_ids_to_tokens(x) for x in ids])
```

교재 연결

설명 포인트: 출력 토큰을 예쁘게 보여주는 데서 끝내지 말고, 모델별 평균 토큰 수·p95·숫자/영문/고유명사 분절을 표로 비교한다.

TF-IDF 기준선 코드 읽기

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vec = TfidfVectorizer(
    tokenizer=my_korean_tokenize,
    token_pattern=None,
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True,
)
X = vec.fit_transform(documents)          # [N, |V|], sparse
q = vec.transform([query])                # [1, |V|]
scores = cosine_similarity(q, X)[0]
```

실습

shape 질문: $X.nnz / (X.shape[0] * X.shape[1])$ 은 무엇을 측정하는가?

최신 동향: sparse·dense·multi-vector를 한 모델에

BGE-M3는 하나의 모델에서 세 검색 표현을 지원하도록 설계됐다.

- **dense** — 문장/문서 단일 벡터
- **sparse** — 학습된 lexical weight
- **multi-vector** — 토큰 수준 late interaction

논문 보고 범위: 100개 이상 언어, 최대 8,192 토큰 입력. 그러나 실제 선택은 자신의 한국어·도메인 평가로 결정한다.

출처: Chen et al., “BGE M3-Embedding,” Findings of ACL 2024 · [ACL Anthology](#) · [arXiv:2402.03216](#)

논문 도판: M3는 세 가지 “다중성”을 묶는다

그림: 다국어·다기능·다중 입자도를 하나의 embedding model에서 지원하려는 BGE-M3의 설계 범위.

교재 연결

Chapter 2의 tokenizer·dense embedding과 Chapter 8의 semantic search를 연결하면, 토큰화가 **sparse weight**와 **multi-vector 표현**에도 영향을 준다는 점이 드러난다.

출처: Chen et al. (2024), “M3-Embedding,” Figure 1 · [ACL Anthology](#)

실험 설계: tokenizer × 표현

같은 30개 문서·10개 쿼리에 대해 비교한다.

실험군	tokenizer	표현	측정
A	whitespace	TF-IDF unigram	Recall@5, vocab, sparsity
B	형태소/사용자 사전	TF-IDF 1-2gram	동일
C	모델 subword	dense embedding	Recall@5, 토큰 수, 시간
D	B + C	RRF hybrid	Recall@5, 오류 유형

주의

하이퍼파라미터를 test set에서 고르면 평가 누수다. train/validation/test 역할을 구분한다.

수업 활동: 분절이 답을 바꾸는 순간

1. 팀마다 고유명사·부정·숫자·신조어가 포함된 한국어 문장 5개를 만든다.
2. 두 tokenizer의 토큰과 ID를 비교한다.
3. TF-IDF 상위 특성 10개를 확인한다.
4. “좋은 분절”을 토큰 모양이 아니라 검색 결과로 판정한다.

실습

제출 한 문장: “Tokenizer A는 __ 때문에 토큰 수는 줄었지만/늘었지만, __ 쿼리에서 Recall@5가 __했다.”

형성평가

1. subword가 OOV 문제를 완화하는 원리는?
2. TF-IDF에서 모든 문서에 등장하는 단어의 IDF는 왜 작아지는가?
3. BM25가 TF-IDF에 추가한 두 핵심 보정은?
4. 희소와 밀집 검색을 결합할 때 raw score 합이 위험한 이유는?
5. tokenizer 비교에서 평균 토큰 수 외에 필요한 두 지표는?

교재 연결

Notebook: [../notebooks/week02.ipynb](#) · tokenizer 결과와 sparse matrix를 모두 저장해 재현성을 확보한다.

핵심 정리

- 토큰화는 모델이 관측하는 단위와 계산 비용을 동시에 정한다.
- TF-IDF/BM25는 정확한 lexical evidence를 보존하는 강한 기준선이다.
- 밀집 표현은 lexical gap을 줄이지만 부정·코드·최신 고유명사를 놓칠 수 있다.
- hybrid는 두 오류 유형을 결합하되 validation으로 결합 규칙을 정한다.
- 최신 모델도 희소 표현을 버리기보다 통합하는 방향으로 발전한다.

핵심

다음 주: 단어의 의미를 **주변 단어로부터 학습**한다.

참고문헌과 원본 연결

- Sennrich et al. (2016), “Neural Machine Translation of Rare Words with Subword Units.” [ACL Anthology](#)
- Kudo (2018), “Subword Regularization.” [ACL Anthology](#)
- Robertson & Zaragoza (2009), “The Probabilistic Relevance Framework: BM25 and Beyond.” [DOI](#)
- Chen et al. (2024), “BGE M3-Embedding.” [ACL Anthology](#)
- Hands-On Large Language Models, Chapter 2. [upstream](#)